

# PYTHON MASTER GUIDE

*Повний довідник Python-розробника & QA-інженера*

Python · Playwright · pytest · TypeScript · Allure · Page Object Model

## □ РОЗДІЛ 1

# Python Basics & Memory Model

*Об'єкти, змінні, типи, рядки — як Python насправді працює в пам'яті*

Python не зберігає значення у змінній — він зберігає посилання на об'єкт у пам'яті. Ця різниця пояснює 80% «дивної» поведінки: чому `is` та `==` дають різні результати, чому список «копіюється» несподівано, чому мутабельні default-аргументи — пастка.

## 1.1 Об'єкти, Змінні та Пам'ять

### □ АНАЛОГІЯ

Місто (RAM). Будинок (об'єкт) — за конкретною адресою. Змінна — папірець з адресою, не сам будинок. `a=b` копіює ПАПІРЕЦЬ: обидва вказують на ОДИН будинок. GC видаляє будинок лише коли всі папірці загублені.

```
a = 'hello'

print(type(a)) # <class 'str'>

print(id(a)) # 140234567890 ← адреса об'єкта в RAM

# Присвоєння = прив'язка імені, НЕ копіювання

b = a

print(a is b) # True — ОДИН об'єкт!

# is vs ==

x = [1,2,3]; y = [1,2,3]

print(x == y) # True — рівні ЗНАЧЕННЯ

print(x is y) # False — різні ОБ'ЄКТИ в RAM

# Динамічна типізація: тип — у ОБ'ЄКТИ, не у змінній

var = 42 # var → int

var = 'Python' # var → str (старий int видасть GC)
```

```
var = [1,2,3] # var → list
```

### ❏ GOTCHA & INTERVIEW

INTERVIEW: Python кешує числа -5..256 та короткі рядки (інтернування).  $a=256; b=256$ ;  $a \text{ is } b \rightarrow \text{True}$ . Але  $a=257; b=257$ ;  $a \text{ is } b \rightarrow$  може бути False! Завжди  $==$  для порівняння значень,  $\text{is}$  — ТІЛЬКИ для None.

## 1.2 Мутабельність та Імутабельність

| Тип       | Імутабельні                  | Мутабельні                 |
|-----------|------------------------------|----------------------------|
| Приклади  | int, float, str, tuple, bool | list, dict, set, bytearray |
| Зміна     | Створює НОВИЙ об'єкт         | Змінює існуючий об'єкт     |
| dict key? | ❏ Так (hashable)             | ❏ Ні (unhashable type)     |

```
# ПАСТКА: мутабельний default аргумент

def bad(item, lst=[]): # ❏ lst — ОДИН об'єкт для всіх викликів!

    lst.append(item); return lst

print(bad(1)) # [1]

print(bad(2)) # [1, 2] ← несподівано!

def good(item, lst=None): # ❏ Правильно

    if lst is None: lst = []

    lst.append(item); return lst
```

### ❏ GOTCHA & INTERVIEW

INTERVIEW: Чому list не може бути ключем dict? — list мутабельний, немає `__hash__()`. dict вимагає: хеш не змінюється після вставки. Tuple імутабельний → hashable → може бути ключем.

## 1.3 Рядки та Зрізи

### ❏ АНАЛОГІЯ

Рядок — нитка намистин. Зріз — ножиці: повертають НОВУ нитку, оригінал незмінний. Формула: `[START : STOP(не включно) : STEP]`.

```
s = 'Python rocks!'
```

```
print(s[0]) # 'P' (з початку)

print(s[-1]) # '!' (з кінця)

print(s[0:6]) # 'Python'

print(s[7:]) # 'rocks!'

print(s[::-1]) # '!skcor nohtyP' — реверс рядка

print(s[::2]) # 'Pto ok' — кожен 2-й символ

# f-strings (Python 3.6+) — найпотужніший формат
name = 'AQA'; score = 99.5

print(f'Привіт, {name}! Результат: {score:.1f}%')

print(f'2**10 = {2**10}') # виконує вирази!

# Методи (повертають НОВИЙ рядок — str імутабельний!)

print(' User@Ex.COM '.strip().lower()) # 'user@ex.com'

print('a,b,c'.split(',')) # ['a','b','c']

print('-'.join(['2024','01','15'])) # '2024-01-15'

print('test.py'.endswith('.py')) # True
```

### ❏ GOTCHA & INTERVIEW

INTERVIEW: Конкатенація в циклі через + —  $O(n^2)$ ! Кожен раз новий об'єкт. Правильно: збирай у list, потім `".join(parts)"` —  $O(n)$ .

## □ РОЗДІЛ 2

# Структури Даних & Big O

*List, Dict, Tuple, Set + Алгоритмічна складність — з  
«Грокаємо Алгоритми»*

## 2.1 Big O Notation

## □ АНАЛОГІЯ

Шукаєш слово: гортати всі —  $O(n)$ . Відкрити посередині і ділити навпіл —  $O(\log n)$ . Знати точну сторінку (хеш) —  $O(1)$ . 240 000 слів → бінарний пошук: 18 кроків!

| Big O         | Назва         | Приклади Python             | Пояснення                   |
|---------------|---------------|-----------------------------|-----------------------------|
| $O(1)$        | Константний   | dict[k], set пошук, list[i] | Завжди однакова швидкість □ |
| $O(\log n)$   | Логарифмічний | binary_search, bisect       | Ділить задачу навпіл        |
| $O(n)$        | Лінійний      | for loop, 'in' list         | Пропорційно n               |
| $O(n \log n)$ | Лінійно-лог.  | sorted(), merge sort        | Ефективне сортування        |
| $O(n^2)$      | Квадратичний  | Вкладені цикли              | Повільно! △                 |

## 2.2 Бінарний Пошук — $O(\log n)$

```
def binary_search(lst, target):  
  
    """ $O(\log n)$  час,  $O(1)$  пам'ять. Вимога: список ВІДСОРТОВАНИЙ!"""  
  
    low, high = 0, len(lst) - 1  
  
    while low <= high:  
  
        mid = (low + high) // 2  
  
        if lst[mid] == target: return mid  
  
        elif lst[mid] > target: high = mid - 1  
  
        else: low = mid + 1  
  
    return None
```

```
nums = [1, 3, 5, 7, 9, 11, 13]

print(binary_search(nums, 7)) # 3

print(binary_search(nums, 100)) # None
```

### ❗ GOTCHA & INTERVIEW

Чому не працює на невідсортованому? — Алгоритм припускає: якщо  $mid < target \rightarrow$  ВСЕ зліва теж менше. Для невідсортованого — некоректні результати БЕЗ помилки!

## 2.3 List, Dict, Tuple, Set — Операції та Складність

| Операція           | list          | dict     | set      | tuple  |
|--------------------|---------------|----------|----------|--------|
| Доступ [i/key]     | $O(1)$        | $O(1)$   | N/A      | $O(1)$ |
| Пошук (in)         | $O(n)$        | $O(1)$   | $O(1)$   | $O(n)$ |
| Вставка в кінець   | $O(1)^*$      | $O(1)^*$ | $O(1)^*$ | N/A    |
| Вставка на початок | $O(n)$        | $O(1)$   | $O(1)$   | N/A    |
| Видалення з кінця  | $O(1)$        | $O(1)$   | $O(1)$   | N/A    |
| sort()             | $O(n \log n)$ | N/A      | N/A      | N/A    |

\*  $O(1)$  амортизований — іноді  $O(n)$  при перерозподілі пам'яті

```
# LIST

fruits = ['apple', 'banana', 'cherry']

fruits.append('date') #  $O(1)$  — в кінець

fruits.insert(1, 'avocado') #  $O(n)$  — в середину (зсув!)

fruits.pop() #  $O(1)$  — з кінця

print('apple' in fruits) #  $O(n)$  — лінійний пошук!

# ПАСТКА: [[0]*3]*3 → 3 посилання на ОДИН рядок!

matrix = [[0]*3 for _ in range(3)] # ❌ правильно

# DICT

p = {'name': 'Alice', 'age': 30}

print(p.get('phone', 'N/A')) #  $O(1)$  безпечний доступ

print('name' in p) #  $O(1)$ ! (vs list:  $O(n)$ )
```

```
merged = {'x': 1} | {'x': 2, 'y': 3} # Python 3.9+

# SET

a = {1,2,3,4}; b = {3,4,5,6}

print(a | b) # {1,2,3,4,5,6} – об'єднання

print(a & b) # {3, 4} – перетин

print(a - b) # {1, 2} – різниця

unique = list(set(['a','b','a'])) # видалити дублікати

# TUPLE розпакування

first, *rest = [1, 2, 3, 4, 5] # first=1, rest=[2,3,4,5]

from collections import namedtuple

Point = namedtuple('Point', ['x', 'y'])

p = Point(3, 4); print(p.x, p[0]) # 3 3
```

#### ❏ GOTCHA & INTERVIEW

{ } → порожній DICT, не set! set() → порожній set. frozenset → імутабельний, може бути ключем dict. dict впорядкований з Python 3.7+. collections.defaultdict та Counter — корисні підкласи.

 РОЗДІЛ 3

# Функції — Будівельні Блоки Python

*Scope, LEGB, \*args/\*\*kwargs, Closures, Decorators, Functional Programming*

## 3.1 LEGB Rule — Область Видимості

### □ АНАЛОГІЯ

LEGB — концентричні кімнати. Шукаєш ключ: Local → Enclosing → Global → Built-in (print, len...). Знайдено → зупиняємось. Внутрішня ЗАВЖДИ має перевагу над зовнішньою.

```
x = 'global'

def outer():
    x = 'enclosing'

    def inner():
        x = 'local'

    print(x) # 'local' ← Local перемагає

    inner(); print(x) # 'enclosing'

    outer(); print(x) # 'global'

# global: змінити global з функції

counter = 0

def inc():
    global counter # без цього → UnboundLocalError!

    counter += 1

# nonlocal: змінити enclosing (для замикань)

def make_counter():
    count = 0
```



```
def up():  
  
    nonlocal count  
  
    count += 1; return count  
  
    return up  
  
c = make_counter()  
  
print(c(), c(), c()) # 1 2 3
```

#### ❏ GOTCHA & INTERVIEW

global та nonlocal потрібні щоб ПИСАТИ у зовнішній scope — читати можна без них. Без global: counter+=1 → UnboundLocalError (Python вважає counter локальною і не знаходить перед читанням).

## 3.2 Замикання (Closures) та Декоратори

#### ❏ АНАЛОГІЯ

Замикання — торговий автомат після налаштування: зачинений, але 'пам'ятає' ціну (enclosing). Декоратор — захисний чохол: телефон (функція) той самий, чохол додає логування/кешування.

```
import functools, time  
  
# Замикання  
  
def make_multiplier(factor):  
  
    def multiplier(n):  
  
        return n * factor # factor 'заморожено' з outer  
  
        return multiplier # НЕ викликаємо — повертаємо функцію!  
  
    double = make_multiplier(2) # factor=2 'заморожено'  
  
    print(double(5), double(7)) # 10 14  
  
# Базовий декоратор  
  
def timer(func):  
  
    @functools.wraps(func) # ОБОВ'ЯЗКОВО: зберігає __name__, __doc__  
  
    def wrapper(*args, **kw):  
  
        t = time.time()
```

```
result = func(*args, **kw)

print(f'{func.__name__}: {time.time()-t:.4f}c')

return result

return wrapper

@timer

def slow(n): time.sleep(0.1); return n * 2

print(slow(5)) # slow: 0.1001c → 10

# Декоратор з параметрами (фабрика декораторів)

def repeat(times):

    def decorator(func):

        @functools.wraps(func)

        def wrapper(*a, **kw):

            for _ in range(times): result = func(*a, **kw)

            return result

        return wrapper

    return decorator

@repeat(3)

def greet(name): print(f'Hello, {name}!')

greet('AQA') # виводить тричі

# @lru_cache – мемоізація

from functools import lru_cache

@lru_cache(maxsize=None)

def fib(n):

    if n < 2: return n

    return fib(n-1) + fib(n-2)

print(fib(50)) # миттєво! Без cache:  $O(2^n)$  → з:  $O(n)$ 
```

**❏ GOTCHA & INTERVIEW**

@functools.wraps ОБОВ'ЯЗКОВИЙ! Без нього wrapper.\_\_name\_\_='wrapper' — ламає логування і pytest. Ланцюжок: @A @B def f() → A(B(f)). Виконуються знизу вгору.

### 3.3 Lambda, map, filter, partial — Функціональний стиль

```
# Lambda — для коротких одноразових функцій

square = lambda x: x**2

is_even = lambda x: x % 2 == 0

# map → ITERATOR (lazy, одноразовий)

nums = [1, 2, 3, 4, 5]

squared = list(map(lambda x: x**2, nums)) # [1,4,9,16,25]

# filter → ITERATOR

evens = list(filter(lambda x: x%2==0, nums)) # [2, 4]

# Сучасний стиль (читабельніше ніж map/filter)

result = [x**2 for x in nums if x % 2 == 0] # [4, 16]

# partial — заморозити аргументи

from functools import partial

def power(base, exp): return base ** exp

square = partial(power, exp=2)

cube = partial(power, exp=3)

print(square(5), cube(3)) # 25 27

# zip — 'застібка', зупиняється на найкоротшому

names = ['Alice', 'Bob']

scores = [100, 85, 90]

for n, s in zip(names, scores): # 'Alice':100, 'Bob':85

    print(f'{n}: {s}')
```

## □ РОЗДІЛ 4

# ООП — Об'єктно-Орієнтоване Програмування

*Класи, Наслідування, Інкапсуляція, Поліморфізм, Dunder методи*

## 4.1 Класи, Атрибути, @property

### □ АНАЛОГІЯ

Клас — форма для пряників. Об'єкт — конкретний пряник. Атрибути — начинка. Методи — з'їсти(). `__init__` — момент виходу пряника з форми.

```
class BankAccount:

    bank_name = 'PyBank' # Атрибут КЛАСУ — спільний для всіх

    _total = 0 # '_' = захищений (угода)

    def __init__(self, owner: str, balance: float = 0.0):

        self.owner = owner # Атрибут ЕКЗЕМПЛЯРА

        self.__balance = balance # '__' = name mangling

        BankAccount._total += 1

    def deposit(self, amount: float) -> None:

        if amount <= 0: raise ValueError('Сума > 0')

        self.__balance += amount

    @property

    def balance(self) -> float: return self.__balance # getter

    @balance.setter

    def balance(self, v: float) -> None:
```

```
if v < 0: raise ValueError('Не від'ємний')

self.__balance = v

@classmethod # отримує cls, може створювати підкласи
def get_total(cls) -> int: return cls._total

@staticmethod # не знає про клас і self
def is_valid(a) -> bool:

    return isinstance(a, (int, float)) and a > 0

def __str__(self): return f'Account({self.owner}: {self.__balance:.2f})'

def __repr__(self): return f'BankAccount({self.owner!r},{self.__balance!r})'
```

#### ❏ GOTCHA & INTERVIEW

@staticmethod vs @classmethod: staticmethod — звичайна функція в namespace класу (без self/cls). classmethod отримує cls → може створювати підкласи. Використовуй classmethod як альтернативний конструктор: User.from\_dict(), User.from\_csv().

## 4.2 Наслідування, MRO, ABC, Поліморфізм

```
class Animal:

    def __init__(self, name: str, sound: str):

        self.name = name; self.sound = sound

    def speak(self) -> str: return f'{self.name}: {self.sound}!'

class Dog(Animal):

    def __init__(self, name: str, breed: str):

        super().__init__(name, 'Гав') # виклик батьківського __init__

        self.breed = breed

    def speak(self) -> str: # перевизначення

        return f'{self.name} ({self.breed}): Гав-Гав!'

# MRO — порядок пошуку методів (C3 linearization)

print(Dog.__mro__) # Dog → Animal → object
```

```
# Множинне наслідування

class Swimmable: def swim(self): return f'{self.name} плаває'

class Flyable: def fly(self): return f'{self.name} літає'

class Duck(Animal, Swimmable, Flyable):

def __init__(self, name): super().__init__(name, 'Кря')

# Перевірка ієрархії

dog = Dog('Rex', 'Лабрадор')

print(isinstance(dog, Animal)) # True — IS-A відношення

# Абстрактний клас — ABC

from abc import ABC, abstractmethod

class Shape(ABC):

@abstractmethod

def area(self) -> float: pass

def info(self): return f'{type(self).__name__}: S={self.area():.2f}'

class Circle(Shape):

def __init__(self, r): self.r = r

def area(self): import math; return math.pi * self.r ** 2

# Поліморфізм: одна функція — різна поведінка

shapes = [Circle(5), Circle(3)]

for s in shapes: print(s.info()) # кожен клас — своя реалізація!
```

#### ❏ GOTCHA & INTERVIEW

`__str__` vs `__repr__`: `__str__` для юзера (print). `__repr__` для розробника (debug), ідеально `eval(repr(obj))` відтворює об'єкт. Якщо тільки `__repr__` — використовується і для `str()`. `__new__` СТВОРЮЄ об'єкт, `__init__` ІНІЦІАЛІЗУЄ вже створений.

 РОЗДІЛ 5

# Ітератори, Генератори та Винятки

*yield, with statement, try/except/else/finally, власні exceptions*

## 5.1 Протокол Ітерації та Генератори

### ■ АНАЛОГІЯ

Генератор — кран, не ванна. `next()` → одна порція. Ванна (`list`) — весь об'єм одразу. 1 мільярд рядків: генератор читає по одному, `list` вибухне.

```
# Протокол ітерації: for = iter() + next() автоматично

lst = [1, 2, 3]

it = iter(lst) # lst.__iter__()

print(next(it)) # 1 — it.__next__()

# Генераторна функція — yield 'заморожує' виконання

def fib_gen():

    a, b = 0, 1

    while True:

        yield a # заморожуємо і повертаємо a

        a, b = b, a + b # відновлюємо після yield

f = fib_gen()

print([next(f) for _ in range(10)]) # [0,1,1,2,3,5,8,13,21,34]

# Generator expression — lazy, ~120 bytes vs 8MB+ RAM

# list: [x**2 for x in range(10**6)] — 8MB+ в RAM

# gen: (x**2 for x in range(10**6)) — ~120 bytes

gen = (x**2 for x in range(10**6))

print(next(gen)) # 0 — лише перше обчислено!
```

```
# yield from – делегування ітерації

def chain(*iters):

    for it in iters: yield from it

print(list(chain([1,2],[3,4]))) # [1,2,3,4]
```

#### ❏ GOTCHA & INTERVIEW

Generator — ОДНОРАЗОВИЙ: пройшов усі значення → StopIteration. Для повторного обходу: list() або новий генератор. Generator expression (x for x) vs list [x for x]: lazy vs eager.

## 5.2 Контекст-менеджери (with)

#### ❏ АНАЛОГІЯ

with — адміністратор залу: \_\_enter\_\_ відкриває, тіло виконується, \_\_exit\_\_ ЗАВЖДИ прибирає — навіть якщо виняток.

```
# Власний контекст-менеджер через @contextmanager

from contextlib import contextmanager

import time

@contextmanager

def timer(label):

    t = time.time()

    try:

        yield # тіло with виконується тут

    finally: # ГАРАНТОВАНЕ виконання

        print(f'{label}: {time.time()-t:.4f}s')

with timer('Тест'): # Тест: 0.1001s

    time.sleep(0.1)
```

## 5.3 Винятки та їх Обробка

```
# Ієрархія: BaseException → Exception → ValueError / TypeError / ...
```



```
def safe_divide(a, b):  
  
    try:  
  
        result = a / b  
  
    except ZeroDivisionError:  
  
        return float('inf')  
  
    except TypeError as e:  
  
        raise # перекинути далі  
  
    else:  
  
        return result # ТІЛЬКИ якщо не було винятку  
  
    finally:  
  
        print('Завжди') # ЗАВЖДИ, навіть при return!  
  
# Власні винятки  
  
class AppError(Exception):  
  
    def __init__(self, msg, code=500):  
  
        super().__init__(msg); self.code = code  
  
class ValidationError(AppError):  
  
    def __init__(self, field, msg):  
  
        super().__init__(f'{field}: {msg}', code=400)  
  
        self.field = field  
  
    try:  
  
        raise ValidationError('email', 'невалідний')  
  
    except ValidationError as e:  
  
        print(f'[{e.code}] {e}') # [400] email: невалідний
```

### ❏ GOTCHA & INTERVIEW

НИКОЛИ 'except:' або 'except Exception:' без re-raise — це 'ковтання' помилок. assert НЕ для production-валідації: python -O вимикає всі assert. except (Type1, Type2) as e: — обробка кількох типів разом.

## □ РОЗДІЛ 6

# Тестування Python — AQA Engineer Focus

*unittest · pytest · Mock · Fixtures · Parametrize — повний курс*

## 6.1 unittest — Стандартна Бібліотека

### □ АНАЛОГІЯ

unittest — офіційна кухня. setUp() — підготовка інгредієнтів. tearDown() — прибирання. setUpClass() — підготовка кухні на весь день.

```
import unittest

def divide(x, y):
    if y == 0: raise ValueError('Ділення на нуль!')
    return x / y

class TestDivide(unittest.TestCase):

    @classmethod
    def setUpClass(cls): cls.data = [(10,2,5.0),(9,3,3.0)]

    def setUp(self): self.x = 10; self.y = 5

    def tearDown(self): pass

    def test_basic(self):
        self.assertEqual(divide(10, 5), 2.0)
        self.assertIsInstance(divide(10, 5), float)

    def test_zero(self):
        with self.assertRaises(ValueError) as ctx:
```

```
divide(10, 0)

self.assertIn('нуль', str(ctx.exception))

def test_asserts(self):

self.assertTrue(5 > 3) # bool(x) is True

self.assertFalse(3 > 5) # bool(x) is False

self.assertIsNone(None) # x is None

self.assertIn('py', 'python') # a in b

self.assertGreater(10, 5) # a > b

self.assertAlmostEqual(0.1+0.2, 0.3, places=10)

self.assertListEqual([1,2,3], [1,2,3])

self.assertDictEqual({'a':1}, {'a':1})

@unittest.skip('Тимчасово вимкнено')

def test_skip(self): self.fail('Не має виконуватись')

@unittest.expectedFailure

def test_known_bug(self): self.assertEqual(1, 2)
```

#### ❏ GOTCHA & INTERVIEW

Порядок lifecycle: setUpClass → (setUp → test\_X → tearDown) × N → tearDownClass. setUp/tearDown: ізоляція тестів (кожен незалежний). setUpClass: дорогі операції (відкрити БД один раз).

## 6.2 Mock — Ізоляція Залежностей

#### ❏ АНАЛОГІЯ

Mock — кіно-дублер. Справжня зірка (реальна БД, API) — дорога і непередбачувана. Дублер (Mock) виконує ті самі 'рухи' безпечно. @mock.patch — режисер, що підмінює зірку на час тесту.

```
import unittest

import unittest.mock as mock

def send_email(to, subj, body): return True # зовнішня залежність!

class UserService:
```

```
def register(self, email, name):

    user = {'email': email, 'name': name}

    send_email(email, f'Ласкаво, {name}!', 'OK')

    return user


class TestUserService(unittest.TestCase):

    @mock.patch('__main__.send_email') # підміняємо функцію

    def test_register_sends_email(self, mock_send):

        svc = UserService()

        user = svc.register('t@t.com', 'Alice')

        self.assertEqual(user['name'], 'Alice')

        mock_send.assert_called_once_with(

            't@t.com', 'Ласкаво, Alice!', 'OK'

        )

    def test_mock_attrs(self):

        m = mock.MagicMock()

        m.calc.return_value = 42

        self.assertEqual(m.calc(1, 2), 42)

        m.calc.assert_called_with(1, 2)

        self.assertEqual(m.calc.call_count, 1)

        # side_effect – кинути виняток

        m.risky.side_effect = ValueError('Ой!')

        with self.assertRaises(ValueError): m.risky()

        # reset_mock – скинути лічильник

        m.calc.reset_mock()

        self.assertFalse(m.calc.called)

        # side_effect як список – послідовні значення

        m.seq.side_effect = [1, 'два', ValueError]
```

```
print(m.seq(), m.seq()) # 1 'два'
```

#### ❏ GOTCHA & INTERVIEW

return\_value vs side\_effect: return\_value — завжди цей результат. side\_effect — якщо виняток → кидає, якщо функція → викликає, якщо список → по черзі. side\_effect перекриває return\_value.

## 6.3 pytest — Сучасний Підхід

#### ❏ НАВИЩО ЦЕ?

pytest — де-факто стандарт. Простіший синтаксис (assert vs self.assertEqual), fixtures > setUp/tearDown, параметризація, плагіни. Зворотньо сумісний з unittest.

```
import pytest

# Базовий тест — просто assert, без class

def test_divide(): assert 10/2 == 5.0

# Fixtures — повторно використовуваний setup/teardown

@pytest.fixture

def db(): # scope='function' за замовчуванням

    conn = 'connect_to_test_db()'

    yield conn # тіло тесту виконується тут

    pass # teardown: закрити з'єднання

@pytest.fixture(scope='session') # один раз на всю сесію

def browser():

    b = 'launch_browser()'

    yield b

    b = 'browser.close()'

def test_query(db): assert db == 'connect_to_test_db()'

# Параметризація — один тест, кілька сценаріїв

@pytest.mark.parametrize('a,b,expected', [

    (10, 2, 5.0),
```

```

(9, 3, 3.0),
(-6, 2, -3.0),
], ids=['10/2', '9/3', '-6/2'])

def test_divide_param(a, b, expected):
    assert a / b == expected

# Маркери

@pytest.mark.skip(reason='Не реалізовано')

def test_future(): pass

@pytest.mark.xfail(reason='Відомий баг #42')

def test_known_bug(): assert 1 == 2

# mocker (pip install pytest-mock)

def test_with_mocker(mocker):

    m = mocker.patch('module.function')

    m.return_value = 999

    from module import function

    assert function() == 999

    m.assert_called_once()

```

| Команда pytest                       | Опис                                |
|--------------------------------------|-------------------------------------|
| pytest                               | Всі тести                           |
| pytest -v                            | Verbose — детальний вивід           |
| pytest -x                            | Стоп на першій помилці              |
| pytest tests/test_api.py::test_login | Конкретний тест                     |
| pytest -k 'login or register'        | За ключовим словом                  |
| pytest -m smoke                      | За маркером                         |
| pytest --tb=short                    | Короткий traceback                  |
| pytest --cov=src --cov-report=html   | HTML coverage звіт                  |
| pytest -n 4                          | 4 паралельних потоки (pytest-xdist) |

|                |                           |
|----------------|---------------------------|
| pytest -s      | Показати print() виводи   |
| pytest --co -q | Список тестів без запуску |

## □ РОЗДІЛ 7

# Playwright + Page Object Model

*Веб-автоматизація, POM архітектура, Allure, Fixtures — повний AQA стек*

## 7.1 Чому Playwright?

| Критерій          | Playwright                    | Selenium                    |
|-------------------|-------------------------------|-----------------------------|
| Швидкість         | ⚡ Дуже швидкий                | □ Повільніший               |
| Очікування        | □ Вбудований auto-wait        | □ Ручні waits (time.sleep!) |
| Браузери          | Chrome, Firefox, Safari       | Те саме, але складніше      |
| Мобільна емуляція | □ Вбудована                   | □ Складно                   |
| Скріншоти/відео   | □ З коробки                   | □ Бібліотеки                |
| Mock API/мережа   | □ Вбудований                  | □ Немає                     |
| Встановлення      | 1 команда: playwright install | Потрібні драйвери           |

## 7.2 Архітектура Проекту (Page Object Model)

## □ НАВИЩО ЦЕ?

POM — шаблон: кожна сторінка = клас. Селектори і дії — в pages/. Тести — лише бізнес-логіка, без CSS/XPath. Якщо на сайті змінився селектор → правиш ОДИН файл, а не всі тести.

```
my_project/
├─ framework/
│ └─ base/
│   │ └─ base_page.py ← Всі дії з браузером (клік, ввід...)
│   │ └─ base_test.py ← Батьківський клас для тестів
│   └─ config/
│     └─ settings.py ← Налаштування (.env → Python об'єкт)
```



```
| └─ helpers/
|
| └─ data_generator.py ← Faker: генерація даних
|
└─ pages/
    | └─ login_page.py ← Кожна сторінка = окремий файл
    |
    | └─ register_page.py
    |
    └─ tests/
        | └─ conftest.py ← Спільні фікстури (browser, context, page)
        |
        | └─ test_auth/
        |
        | └─ test_login.py
        |
        └─ .env ← BASE_URL, паролі (НЕ КОМІТИТИ!)
            └─ pyproject.toml ← Конфіг pytest + markers
```

## 7.3 BasePage — Серце Стеку

```
# framework/base/base_page.py

import allure

from playwright.sync_api import Page, expect
from framework.config.settings import settings

class BasePage:

    def __init__(self, page: Page):

        self.page = page

        self.page.set_default_timeout(settings.default_timeout)

        @allure.step('Відкрити: {url}')

        def navigate(self, url: str) -> None:

            full = url if url.startswith('http') else f'{settings.base_url}{url}'

            self.page.goto(full)

        @allure.step('Клікнути: {selector}')

        def click(self, selector: str) -> None:

            self.page.locator(selector).click()
```

```
@allure.step('Ввести текст: {selector}')

def fill(self, selector: str, text: str) -> None:

    self.page.locator(selector).fill(text)

def is_visible(self, selector: str) -> bool:

    return self.page.locator(selector).is_visible()

def get_current_url(self) -> str:

    return self.page.url

def screenshot(self, name: str = 'screenshot') -> None:

    data = self.page.screenshot()

    allure.attach(data, name=name,

attachment_type=allure.attachment_type.PNG)
```

```
# pages/login_page.py

import allure

from playwright.sync_api import Page, expect
from framework.base.base_page import BasePage

class LoginPage(BasePage):

    URL = '/sign-in'

    EMAIL_INPUT = "input[name='email']"
    PASS_INPUT = "input[name='password']"
    SUBMIT_BTN = "button[type='submit']"
    ERROR_MSG = ".error-message"

    def __init__(self, page: Page):

        super().__init__(page)

    @allure.step('Відкрити сторінку логіну')

    def open(self) -> 'LoginPage':

        self.navigate(self.URL)
```

```
return self

 allure.step('Ввести email: {email}')

 def enter_email(self, email: str) -> None:

    self.fill(self.EMAIL_INPUT, email)

 allure.step('Ввести пароль')

 def enter_password(self, password: str) -> None:

    self.fill(self.PASS_INPUT, password)

 allure.step('Натиснути Увійти')

 def click_submit(self) -> None:

    self.click(self.SUBMIT_BTN)

 def login(self, email: str, password: str) -> None:

    self.enter_email(email)

    self.enter_password(password)

    self.click_submit()

 allure.step('Перевірити: логін успішний')

 def expect_logged_in(self) -> None:

    self.page.wait_for_function(

        "window.location.pathname.includes('dashboard')",

        timeout=15_000

    )

    assert 'dashboard' in self.get_current_url()
```

## 7.4 conftest.py — Фікстури Playwright

```
# tests/conftest.py

import pytest

import allure

from playwright.sync_api import sync_playwright
```

```
from framework.config.settings import settings

from pages.login_page import LoginPage

@pytest.fixture(scope='session')

def playwright_instance():

    with sync_playwright() as pw:

        yield pw

@pytest.fixture(scope='session')

def browser(playwright_instance):

    browser = getattr(playwright_instance, settings.browser).launch(

        headless=settings.headless,

        slow_mo=settings.slow_mo,

    )

    yield browser

    browser.close()

@pytest.fixture(scope='session')

def context(browser):

    ctx = browser.new_context(

        viewport={'width': 1920, 'height': 1080},

        storage_state='auth_state.json' if os.path.exists('auth_state.json') else None

    )

    yield ctx

    ctx.close()

@pytest.fixture

def page(context): # scope='function' – нова вкладка для КАЖДОГО тесту

    p = context.new_page()

    yield p

    p.close()
```

```
@pytest.fixture

def login_page(page):

    return LoginPage(page)

# Autouse – скріншот при будь-якому падінні

@pytest.fixture(autouse=True)

def screenshot_on_failure(request, page):

    yield

    if request.node.rep_call.failed:

        allure.attach(page.screenshot(),

            name='failure_screenshot',

            attachment_type=allure.attachment_type.PNG)
```

## 7.5 Написання Тестів — Allure + POM

```
# tests/test_auth/test_login.py

import pytest

import allure

from pages.login_page import LoginPage

from framework.base.base_test import BaseTest

@allure.epic('Сервіс needmylink')

@allure.feature('Авторизація')

class TestLogin(BaseTest):

    @allure.story('Успішний логін')

    @allure.severity(allure.severity_level.BLOCKER)

    @allure.tag('smoke', 'auth')

    def test_valid_login(self, login_page: LoginPage):

        """Юзер входить з правильними даними → редірект на /dashboard."""

        with allure.step('1. Відкрити сторінку логіну'):

            login_page.open()
```

```

with allure.step('2. Ввести валідні credentials'):

    login_page.login('user@test.com', 'TestPass123!')

with allure.step('3. Перевірити редірект'):

    login_page.expect_logged_in()

    @allure.story('Невалідний логін')

    @allure.severity(allure.severity_level.CRITICAL)

    @pytest.mark.parametrize('email,password,expected_error', [

        ('wrong@test.com', 'pass', 'Невірний email або пароль'),

        ('user@test.com', 'wrongpass', 'Невірний email або пароль'),

        ('', '', 'Це поле обов\язкове'),

    ], ids=['wrong-email', 'wrong-pass', 'empty'])

    def test_invalid_login(self, login_page, email, password, expected_error):

        login_page.open()

        login_page.login(email, password)

        login_page.expect_error_message(expected_error)

```

## 7.6 Playwright — Ключові API

```

# Локатори — пошук елементів

page.locator('#email') # CSS за id

page.locator("input[name='email']") # CSS за атрибутом

page.get_by_text('Увійти') # за текстом

page.get_by_placeholder('example@mail') # за placeholder

page.get_by_role('button', name='Submit') # за роллю

page.get_by_test_id('login-btn') # data-testid (найкращий!)

# Дії

loc.fill('text') # ввести текст (очищає спочатку)

loc.click() # клікнути

loc.check() # чекбокс ☐

```

```
loc.press('Enter') # натиснути клавішу

loc.select_option('ua') # select dropdown

# Читання даних

loc.inner_text() # видимий текст

loc.input_value() # значення поля

loc.get_attribute('href') # значення атрибута

loc.is_visible() # True/False

loc.count() # кількість знайдених

# Очікування – Playwright чекає сам!

loc.wait_for(state='visible') # чекати видимості

page.wait_for_url('**/dashboard') # чекати URL

page.wait_for_load_state('domcontentloaded')

page.wait_for_function( # JS умова

"window.location.pathname === '/dashboard'"

)

# expect() – перевірки з auto-wait (до 5 сек)

from playwright.sync_api import expect

expect(loc).to_be_visible()

expect(loc).to_have_text('Привіт')

expect(loc).to_contain_text('Прив')

expect(loc).to_be_enabled()

expect(loc).to_have_count(5)

expect(page).to_have_url('**dashboard**')

expect(page).to_have_title('Головна')
```

## 7.7 Правило вибору селекторів

| Пріоритет | Селектор | Приклад | Надійність |
|-----------|----------|---------|------------|
|-----------|----------|---------|------------|

|               |               |                                      |             |
|---------------|---------------|--------------------------------------|-------------|
| 1 — Найкращий | data-testid   | page.get_by_test_id('btn')           | Максимальна |
| 2             | name атрибут  | input[name='email']                  | Висока      |
| 3             | placeholder   | input[placeholder='Email']           | Висока      |
| 4             | Role + name   | get_by_role('button', name='Submit') | Висока      |
| 5             | Текст кнопки  | button:has-text('Увійти')            | Середня     |
| 6             | href          | a[href='/dashboard']                 | Середня     |
| 7 — Уникай    | CSS class     | .btn-primary                         | Низька      |
| 8 — Ніколи    | Динамічний id | #:r1: #:r2:                          | Нульова     |

#### ▣ AQA FOCUS

Діагностичний скрипт — запускай на кожній новій сторінці: `python3 -c "from playwright.sync_api import sync_playwright; ..."` Отримуєш список всіх `input[name=...]`, `button[text=...]`, `a[href=...]`. Динамічний `id=':r1:'` → НІКОЛИ не використовуй. Краще: попроси розробника додати `data-testid='login-email'`.

## 7.8 Allure — Красиві Звіти

```
import allure

# Ієрархія у звіті:

# Epic → Feature → Story → Step

@allure.epic('Інтернет-магазин') # продукт

@allure.feature('Авторизація') # функціональний блок

class TestLogin:

    @allure.story('Успішний логін') # конкретний сценарій

    @allure.severity(allure.severity_level.BLOCKER)

    @allure.title('Юзер входить з правильними даними')

    @allure.tag('smoke', 'auth')

    def test_valid_login(self):

        with allure.step('1. Відкрити /login'): ...

        with allure.step('2. Ввести credentials'): ...
```



```
with allure.step('3. Перевірити редірект'): ...

# Додати скріншот до звіту
screenshot = page.screenshot()

allure.attach(screenshot, name='Failure screenshot',
attachment_type=allure.attachment_type.PNG)

# Запуск і відкриття звіту
# pytest tests/ -v
# allure serve allure-results
```

| Severity | Коли використовувати             | Приклад                       |
|----------|----------------------------------|-------------------------------|
| BLOCKER  | Без цього додаток не використати | Не можна залогінитись взагалі |
| CRITICAL | Ключова функція зламана          | Не можна додати товар в кошик |
| NORMAL   | Стандартний функціонал           | Не працює сортування          |
| MINOR    | Незручність                      | Неправильний формат дати      |
| TRIVIAL  | Косметика                        | Неправильний відступ          |

 РОЗДІЛ 8

# Конкурентність: asyncio · threading · multiprocessing

*GIL, IO-bound vs CPU-bound, JSON, CSV, SQLite, HTTP requests*

## ⚠ УВАГА / ПАСТКА

GIL (Global Interpreter Lock) — внутрішній м'ютекс CPython: лише один потік Python-байткоду одночасно. Threading не прискорює CPU-bound задачі. Для IO-bound (мережа, файли) — threading або asyncio. Для CPU-bound (обчислення) — multiprocessing.

## 8.1 asyncio — IO-bound (HTTP, WebSocket)

```
import asyncio

import aiohttp # pip install aiohttp

async def fetch(session, url: str, n: int) -> str:

    async with session.get(url) as r:

        print(f'#{n}: {r.status}')

    return await r.text()

async def main():

    urls = ['https://httpbin.org/get'] * 5

    async with aiohttp.ClientSession() as session:

        tasks = [asyncio.create_task(fetch(session, u, i))

        for i, u in enumerate(urls)]

        results = await asyncio.gather(*tasks) # усі паралельно!

        print(f'{len(results)} завантажено')

    asyncio.run(main())

# Ключові концепти:
```

```
# async def – визначає coroutine

# await – передати управління event loop (не блокує!)

# asyncio.run() – запускає event loop (Python 3.7+)

# asyncio.gather() – кілька задач паралельно

# asyncio.create_task() – планує coroutine
```

## 8.2 threading, multiprocessing та порівняння

| Критерій   | asyncio    | threading       | multiprocessing   |
|------------|------------|-----------------|-------------------|
| IO-bound   | □ Найкраще | □ Добре         | □ Зайвий overhead |
| CPU-bound  | □ Ні       | □ GIL блокує    | □ Найкраще        |
| Пам'ять    | □ Мінімум  | △ Спільна       | □ Окремі процеси  |
| Складність | Середня    | Race conditions | Складно debug     |

## 8.3 Сховища Даних: JSON, CSV, SQLite

### △ УВАГА / ПАСТКА

SQL INJECTION: НІКОЛИ не форматуй SQL через % або f-strings. Завжди параметризовані запити: `execute('WHERE id=?', (uid,))` — SQLite екранує автоматично.

```
import json, csv, sqlite3

# JSON

data = {'name': 'Alice', 'skills': ['Python', 'pytest']}

s = json.dumps(data, ensure_ascii=False, indent=2) # → str

d = json.loads(s) # → dict

with open('users.json', 'w') as f: json.dump(data, f)

with open('users.json', 'r') as f: loaded = json.load(f)

# CSV

with open('users.csv', 'r') as f:

    for row in csv.DictReader(f): # кожен рядок як dict

        print(row['name'])
```

```
with open('out.csv', 'w', newline='') as f:

    w = csv.DictWriter(f, fieldnames=['name','age'])

    w.writeheader()

    w.writerows([{'name':'Alice','age':30}])


# SQLite (in-memory для тестів!)

from contextlib import contextmanager

@contextmanager

def get_db(path=':memory:'):

    conn = sqlite3.connect(path)

    conn.row_factory = sqlite3.Row # row['name'] замість row[0]

    try: yield conn; conn.commit()

    except: conn.rollback(); raise

    finally: conn.close()


with get_db() as conn:

    conn.execute('''CREATE TABLE users

(id INTEGER PRIMARY KEY AUTOINCREMENT,

name TEXT, email TEXT UNIQUE)''')

    conn.execute('INSERT INTO users(name,email) VALUES(?,?)',

('Alice','a@t.com')) # ← параметризований!

    row = conn.execute(

'SELECT * FROM users WHERE name=:n',{ 'n':'Alice' }

).fetchone()

    print(dict(row)) # {'id':1,'name':'Alice','email':'a@t.com'}
```

#### ❏ AQA FOCUS

`:memory:` — in-memory БД для тестів, зникає при закритті. Ідеально для ізоляції.  
`conn.row_factory=sqlite3.Row` → `row['name']` замість `row[0]`. `requests.raise_for_status()` кидає `HTTPError` при 4xx/5xx. Завжди встановлюй `timeout`! Без `timeout` — `request` може чекати вічно.

## □ РОЗДІЛ 9

# JavaScript & TypeScript для AQA

*Основи JS/TS для тестувальника що знає Python —  
Playwright на TS*

## □ НАВІЩО ЦЕ?

JS/TS потрібен AQA тому що: Cypress працює ТІЛЬКИ на JS/TS. Playwright офіційно підтримує TS (більшість документації). Більшість фронтенду — React/Vue/Angular (TS). AQA з JS/TS + Python коштує дорожче.

## 9.1 JavaScript vs TypeScript

| Характеристика   | JavaScript                         | TypeScript                          |
|------------------|------------------------------------|-------------------------------------|
| Типізація        | Динамічна (змінна = будь-який тип) | Статична (типи обов'язкові)         |
| Помилки типів    | Runtime (вже у production!)        | Compile-time (до запуску)           |
| Аналог Python    | Python без type hints              | Python з type hints (обов'язковими) |
| Підтримка IDE    | Базова                             | Повна (автодоповнення, рефакторинг) |
| Популярність AQA | Cypress (старі проекти)            | Playwright, Cypress, WebdriverIO    |

## 9.2 Базовий синтаксис — Шпаргалка Python → JS/TS

| Python                       | JavaScript/TypeScript                        | Примітка                                          |
|------------------------------|----------------------------------------------|---------------------------------------------------|
| <code>x = 5</code>           | <code>let x = 5 / const x = 5</code>         | <code>const</code> — не можна перепризначити      |
| <code>x = 'hello'</code>     | <code>let x: string = 'hello'</code>         | TS: тип явно вказаний                             |
| <code>True / False</code>    | <code>true / false</code>                    | Малі літери!                                      |
| <code>None</code>            | <code>null</code> або <code>undefined</code> | <code>undefined</code> — змінна не ініціалізована |
| <code>print('hi')</code>     | <code>console.log('hi')</code>               |                                                   |
| <code>f'Hello {name}'</code> | <code>`Hello \${name}`</code>                | Backtick, не лапки                                |

| # comment          | // comment         |                              |
|--------------------|--------------------|------------------------------|
| [1,2,3]            | [1,2,3]: number[]  | TS: тип масиву               |
| {'a':1}            | {a: 1}             | TS: без лапок навколо ключів |
| def f(x): return x | const f = (x) => x | Arrow function               |
| class Foo:         | class Foo {        | Фігурні дужки замість :      |
| if x == 5:         | if (x === 5) {     | === замість ==, дужки        |

## 9.3 Функції та async/await у JS/TS

```
// TypeScript – аналоги Python конструкцій

// Звичайна функція

function add(a: number, b: number): number {

  return a + b;

}

// Arrow function (стрілкова – найпоширеніша в тестах)

const add = (a: number, b: number): number => a + b;

// async/await (аналог Python asyncio)

async function fetchUser(id: number): Promise<User> {

  const response = await fetch(`/api/users/${id}`);

  const data = await response.json();

  return data;

}

// Типи та інтерфейси

interface User {

  id: number;

  name: string;

  email: string;

  role?: string; // ? = необов'язкове
```

```
}

// Enum

enum Severity { BLOCKER, CRITICAL, NORMAL, MINOR }

// Array методи – аналоги Python

const nums: number[] = [1, 2, 3, 4, 5];

const squared = nums.map(x => x ** 2); // [1,4,9,16,25]

const evens = nums.filter(x => x % 2 === 0); // [2,4]

const sum = nums.reduce((acc, x) => acc+x, 0); // 15

// Деструктуризація (як Python unpacking)

const [first, ...rest] = [1, 2, 3]; // first=1, rest=[2,3]

const {name, age} = user; // як dict unpacking
```

## 9.4 Playwright на TypeScript — Page Object Model

```
// pages/LoginPage.ts

import { Page, Locator, expect } from '@playwright/test';

export class LoginPage {

  readonly page: Page;

  readonly emailInput: Locator;

  readonly passwordInput: Locator;

  readonly submitBtn: Locator;

  constructor(page: Page) {

    this.page = page;

    this.emailInput = page.locator("input[name='email']");

    this.passwordInput = page.locator("input[name='password']");

    this.submitBtn = page.locator("button[type='submit']");

  }

}
```

```
async open(): Promise<void> {

  await this.page.goto('/login');

}

async login(email: string, password: string): Promise<void> {

  await this.emailInput.fill(email);

  await this.passwordInput.fill(password);

  await this.submitBtn.click();

}

async expectLoggedIn(): Promise<void> {

  await expect(this.page).toHaveURL(/dashboard/);

}

}

// tests/login.spec.ts

import { test, expect } from '@playwright/test';

import { LoginPage } from '../pages/LoginPage';

test.describe('Авторизація', () => {

  test('успішний логін', async ({ page }) => {

    const login = new LoginPage(page);

    await login.open();

    await login.login('user@test.com', 'Pass123!');

    await login.expectLoggedIn();

  });

  test.fail('відомий баг', async ({ page }) => {

    // маркер xfail

  });

});
```

**Команда****Опис**



|                                                  |                       |
|--------------------------------------------------|-----------------------|
| <code>npx playwright test</code>                 | Всі тести             |
| <code>npx playwright test --headed</code>        | З видимим браузером   |
| <code>npx playwright test login.spec.ts</code>   | Конкретний файл       |
| <code>npx playwright test --grep 'логі́н'</code> | За регулярним виразом |
| <code>npx playwright show-report</code>          | Відкрити HTML звіт    |
| <code>npx playwright codegen https://...</code>  | Запис дій у тест!     |

## ❏ РОЗДІЛ 10

# Typing, Regex, Модулі та Оточення

*Type hints · Regular expressions · venv · poetry · mypy*

## 10.1 Type Hints — Статична Типізація

```
from typing import Optional, Union, List, Dict, Callable, Any

from typing import TypeVar, NewType

# Базові анотації

def greet(name: str, times: int = 1) -> str:
    return (f'Hello, {name}! ' * times).strip()

# Optional[X] = Union[X, None]

def find_user(uid: int) -> Optional[dict]:
    return {'name': 'Alice'} if uid == 1 else None

# Python 3.10+ синтаксис

def process(v: int | str) -> str: return str(v)

# NewType — семантично різні типи

UserId = NewType('UserId', int)

EmailStr= NewType('EmailStr',str)

# TypeVar — generic функції

T = TypeVar('T')

def first(items: List[T]) -> Optional[T]:
    return items[0] if items else None

# Callable
```

```
def apply(f: Callable[[int,int],int], a:int, b:int) -> int:

    return f(a, b)
```

## 10.2 Регулярні Вирази (re)

```
import re

text = 'Python 3.11 вийшов у 2022 році!'

re.match(r'Python', text).group() # 'Python' (з початку)

re.search(r'\d+\.\d+', text).group() # '3.11' (будь-де)

re.findall(r'\d+', text) # ['3', '11', '2022']

re.sub(r'\d+', 'X', text) # 'Python X.X вийшов у X році!'

# Compile – ефективніше при повторному використанні

EMAIL = re.compile(

    r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'

)

print(bool(EMAIL.match('user@ex.com'))) # True

print(bool(EMAIL.match('not-email'))) # False

# Групи захоплення

m = re.match(r'(\d{4})-(\d{2})-(\d{2})', '2024-01-15')

year, month, day = m.groups() # ('2024', '01', '15')

# Іменовані групи

m = re.match(r'(?P<year>\d{4})-(?P<month>\d{2})', '2024-01')

print(m.group('year')) # '2024'
```

## 10.3 venv, pip та Poetry

```
# — venv (вбудований) —————

python -m venv .venv

source .venv/bin/activate # Linux/Mac

# .venv\Scripts\activate # Windows
```

```
deactivate # вимкнути

# — pip —————

pip install requests # встановити
pip install requests==2.31.0 # конкретна версія
pip install -r requirements.txt # з файлу
pip freeze > requirements.txt # зберегти поточні

# — Poetry (сучасний підхід) —————

poetry new my-project # новий проект
poetry add requests # залежність
poetry add pytest --group dev # dev залежність
poetry install # з pyproject.toml

# — mypy (статична перевірка типів) —————

pip install mypy
mypy src/ # перевірити
mypy --strict src/ # суворий режим

# — Форматування —————

pip install black isort flake8
black . # автоформатування PEP8
isort . # сортування імпортів
flake8 . # linter
```

## □ РОЗДІЛ 11

# AQA Cheat Sheet & Interview Prep

*Топ-20 питань · Команди · Selector rules · XSS · Щоденний старт*

## 11.1 Топ-20 Interview Питань

### Q1: Що таке GIL і як впливає на threading?

→ GIL — mutex CPython: лише один потік Python-байткоду одночасно. Threading відмінний для IO-bound (GIL відпускається під час чекання IO). Для CPU-bound — multiprocessing.

### Q2: Різниця між is та ==?

→ == порівнює ЗНАЧЕННЯ. is порівнює ІДЕНТИЧНІСТЬ (той самий об'єкт у RAM). is — тільки для None: if x is None.

### Q3: Мутабельний default аргумент — пастка?

→ def f(lst=[]) — lst один раз при визначенні функції. Всі виклики спільно використовують один список. Рішення: def f(lst=None): if lst is None: lst = [].

### Q4: Що таке closure?

→ Функція, що 'пам'ятає' змінні зовнішнього scope після завершення outer(). make\_multiplier(2)  
→ функція де factor=2 'заморожено'.

### Q5: Навіщо @functools.wraps?

→ Без нього wrapper.\_\_name\_\_='wrapper' — ламає логування і pytest. @wraps(func) копіює \_\_name\_\_, \_\_doc\_\_, \_\_module\_\_.

### Q6: Різниця list vs tuple?

→ list мутабельний, більше RAM. tuple імутабельний, менше RAM, може бути ключем dict/set.

### Q7: Чому dict O(1)?

→ Хеш-функція перетворює ключ на числовий індекс масиву. При колізіях →  $O(n)$  (рідко). dict впорядкований з Python 3.7+.

#### Q8: `__str__` vs `__repr__`?

→ `__str__` для юзера (print). `__repr__` для розробника (debug), `eval(repr(obj))` відтворює об'єкт.

#### Q9: Що таке `@property`?

→ Метод, що поводить ся як атрибут. Getter/setter з валідацією. `obj.balance` замість `obj.get_balance()`.

#### Q10: `@staticmethod` vs `@classmethod`?

→ `staticmethod` — без self/cls. `classmethod` отримує cls → може створювати підкласи. Використовуй `classmethod` як альтернативний конструктор.

#### Q11: Generator vs list comprehension?

→ `[x for x]` — список у RAM одразу. `(x for x)` — generator, lazy,  $O(1)$  RAM. Generator одноразовий.

#### Q12: Що таке MRO?

→ Method Resolution Order — C3 linearization. Порядок пошуку методів при множинному наслідуванні. `super()` слідує MRO.

#### Q13: Навіщо `with statement`?

→ Гарантує виконання `__exit__` навіть при винятку. `True` → виняток поглинуто, `False` → пробрасувати.

#### Q14: Що таке Mock?

→ Підробка залежності (БД, API). `@mock.patch` підміняє об'єкт на час тесту. `return_value` — фіксований результат. `side_effect` — виняток або функція.

#### Q15: `pytest fixtures` vs `setUp/tearDown`?

→ Fixtures краще: можна комбінувати, scope (function/class/module/session), yield = teardown. Fixtures можна ін'єктувати в будь-яку функцію.

#### Q16: SQL Injection — як запобігти?

→ НИКОЛИ 'SELECT...' + input або f'...{input}'. Завжди: `execute('WHERE id=?',(uid,))` — SQLite/PostgreSQL екранують автоматично.

#### Q17: POM — Page Object Model?

→ Шаблон: кожна сторінка = клас. Селектори і дії в pages/. Тести — бізнес-логіка. Якщо змінився селектор → правиш ОДИН файл.

#### Q18: Playwright auto-wait?

→ Playwright сам чекає поки елемент стане видимим і клікабельним. НЕ потрібен `time.sleep()`. Використовуй `expect()` для перевірок з `auto-retry`.

#### Q19: asyncio vs threading?

→ `asyncio` — кооперативна однопоточна (`explicit await`). `threading` — витискальна (GIL). `asyncio` краще для IO-bound з тисячами з'єднань.

#### Q20: 'in' — list vs dict/set?

→ `'x in list'` →  $O(n)$  лінійний пошук. `'x in dict/set'` →  $O(1)$  хеш-пошук. Для частого пошуку в великих даних — `dict` або `set`!

## 11.2 Щоденний Старт та Команди Playwright Python

```
# — Щоденний старт —————
cd ~/my_project
source venv/bin/activate # активувати venv
# Рядок змінний має: (venv) ander_sage@host:~$

# — Запуск тестів —————
pytest tests/ -v # всі тести
pytest tests/test_auth/ -v # папка
pytest tests/test_auth/test_login.py -v # файл
pytest ...::TestLogin::test_valid_login -v # один тест
pytest -m smoke -v # за маркером
pytest -m 'not slow' -v # виключити
pytest -x -v # стоп на першій помилці
pytest --tb=short -v # короткий лог
pytest -s -v # показати print()
pytest -n 4 -v # 4 паралельних

# — Allure звіт —————
pytest tests/ -v # запустити тести
allure serve allure-results # відкрити звіт

# — Playwright на TS —————
```

```
npx playwright test # всі тести
npx playwright test --headed # з браузером
npx playwright test login.spec.ts # файл
npx playwright test --grep 'логін' # за regex
npx playwright show-report # HTML звіт
npx playwright codegen https://example.com # запис тесту
```

## 11.3 Типові Помилки та їх Вирішення

| Помилка                                 | Причина                                  | Вирішення                                           |
|-----------------------------------------|------------------------------------------|-----------------------------------------------------|
| ModuleNotFoundError: framework          | Запустив файл напряму через python       | Використовуй: pytest tests/ -v                      |
| ModuleNotFoundError: playwright         | venv не активований                      | source venv/bin/activate                            |
| TimeoutError: waiting for locator       | Елемент не знайдений (змінився селектор) | Запусти діагностичний скрипт                        |
| TimeoutError: waiting for URL           | URL не змінився                          | Використовуй wait_for_function замість wait_for_url |
| AssertionError: очікували X, отримали Y | Перевірка не пройшла                     | Перевір логіку тесту або поведінку сайту            |
| TargetClosedError                       | context фікстура закрилась раніше        | Перевір scope='session' на context                  |

## 11.4 XSS — Безпека для AQA

### ⚠ УВАГА / ПАСТКА

XSS (Cross-Site Scripting) — ін'єкція скрипту на сторінку. Stored XSS: скрипт зберігається у БД. Reflected: у URL. DOM XSS: через JavaScript. Тестуй ТІЛЬКИ на своїх системах! Якщо alert — вразливість підтверджена. Якщо <script> у тексті — захист є.

```
# Базові XSS payload-и для тестування

<script>alert('XSS')</script>

<img src=x onerror=alert('XSS')>

'><script>alert(1)</script>

# Захист у Python

import html
```



```
safe = html.escape("<script>alert('xss')</script>")  
  
print(safe) # &lt;script&gt;alert('xss')&lt;/script&gt;
```

## 11.5 Де що шукати — Quick Reference

| Питання                  | Де дивитись                         |
|--------------------------|-------------------------------------|
| Змінити URL сайту        | .env → BASE_URL=...                 |
| Змінити браузер          | .env → BROWSER=firefox              |
| Показати/сховати браузер | .env → HEADLESS=True/False          |
| Знайти селектор          | F12 у браузері → Inspector          |
| Базові дії з браузером   | framework/base/base_page.py         |
| Селектори сторінки       | pages/login_page.py                 |
| Спільні фікстури         | tests/conftest.py                   |
| Тести                    | tests/test_auth/test_login.py       |
| Генерація даних          | framework/helpers/data_generator.py |
| Запустити тести          | pytest tests/ -v                    |
| Відкрити Allure звіт     | allure serve allure-results         |

---

□ Python Master Guide — Синтезовано з реальних навчальних матеріалів  
Python · Playwright · pytest · TypeScript · Allure · Page Object Model  
AQA Engineer Focus · Junior → Middle Python · 2025